Specifying Types with Literal Types

TypeScript lets you limit the values a string or number variable can have to a set of specific literal values. This *literal type* is not exactly a pure enumeration type, but you can think of it as basically an enumeration type because the behavior is almost identical.

Why would we want to limit the values of a variable? In many cases, we actually have a specific, defined list of values that can be sent to a variable and we'd like to have TypeScript insist on it. For example, in our concert app, tickets have one of five specific states: unsold, held, purchased, refunded, or invalid. On the Rails side, those values are protected with an ActiveRecord and Postgres enum, but we don't have anything like that on the client side.

Let's declare a literal type based on those values. We'll put it in our venue_reducer file:

chapter_13/01/app/javascript/contexts/venue_types.ts

```
export type TicketStatus =
  | "unsold"
  | "held"
  | "purchased"
  | "refunded"
  | "invalid"
```

This declares a new type TicketStatus that can only have the five string values listed. Now, in places that use that value, we replace the declaration of string with TicketStatus, starting right away with the TicketData interface.

Making this change causes one type error in the venue_reducer file, having to do with the return value when we hold the ticket. We need to import the TicketData type and make one further change:

chapter_13/01/app/javascript/contexts/venue_context.ts

```
case "holdTicket": {
```